A Classical Reasoning System for Entailment over Rule Bases, First-Order Logic and Structural Causal Models

Technical report · Artificial Intelligence assignment on knowledge-based entailment checking
NAME · SAUMYADIP DEB

ROLL NO· M26CPS007

1 Introduction and problem statement

A knowledge base (KB) K consists of facts and rules. A query q is entailed by K, written $K \models q$, if q holds in every model of K. The task is binary: the system returns *True* when entailment can be established and *False* otherwise. Consequently, queries whose status is uncertain (neither q nor $\neg q$ is derivable) are labelled False, which is also the natural outcome of a sound but incomplete prover that fails to find a proof within its resource limits.

The overall pipeline is *input* → *internal representation* → *classical reasoning* → *True/False*. Any method may be used to build the representation; the verdict itself is computed exclusively by search, unification, chaining, resolution or SAT procedures. Generative models are not consulted at any stage. Development material comes from three public corpora with quite different character: RuleTaker (synthetic English rule bases with controlled proof depth; Clark et al., 2020), FOLIO (expert-written stories with first-order annotations; Han et al., 2022) and CounterBench (counterfactual questions over small causal graphs). A general system must handle all three and, ideally, unseen variations of them.

2 System overview

The system is organised as a front end (parsers that build a representation), a router and three reasoning engines. Routing inspects the input only: (1) logic supplied in separate premise/conclusion fields, or logical symbols ($\forall$ $\exists$ $\wedge$ $\vee$ $\rightarrow$ $\neg$ $\oplus$, or their ASCII spellings) inside the text, goes to the first-order prover; (2) text containing causal statements with Bernoulli priors or intervention questions goes to the causal-model solver; (3) everything else is read as an English rule base and goes to the chaining engine. Every stage is guarded so that a malformed item yields False instead of aborting a batch, and a single cell of the notebook reproduces the whole pipeline end to end and writes `submission.csv` (columns `id,label`).

Input format	**Internal representation**	**Engine**	**Search / inference**
English facts and if-then rules	Ground atoms + Horn rules (variables allowed)	Chaining engine	Forward chaining (default); backward chaining with BFS / DFS / UCS / A*
Formulas in FOL notation	Clause set (CNF, Skolemised)	First-order prover	Ground DPLL first, then resolution refutation (A* clause selection)
Causal story + intervention question	Boolean structural equations with priors	Causal solver	Exhaustive abduction → action → prediction

Table 1: Routing of inputs to representations and engines.

3 Chosen representation

3.1 Rule bases (RuleTaker-style text)

Sentences are segmented and normalised (articles removed, verbs and plurals reduced to a base form, pronouns and quantifier words such as *someone* mapped to variables). A clause is then stored as a literal $(neg, predicate, args)$, with an argument being a constant or a variable written `?x`. A rule is a pair $(body, head)$ with a conjunctive body of literals and a single head literal. For example, "If someone is smart and kind, then they are successful" becomes $smart(?x) \wedge kind(?x) \rightarrow successful(?x)$. Rules that are stated in several surface templates ("Given that ..., it follows that ...", "Assuming ..., we can say ...", and so on) are mapped to the same internal form. Negation follows the closed-world assumption (negation as failure) and is handled by stratifying the rules so that negated predicates are fully computed before use.

3.2 First-order formulas

Formulas are parsed by a recursive-descent parser with the usual precedence ($\neg$, $\wedge$, $\vee$ and $\oplus$, $\rightarrow$, $\leftrightarrow$) into an abstract syntax tree, then converted to clause normal form: implications, biconditionals and exclusive-or are eliminated, negations are pushed inward (NNF), quantified variables are standardised apart, existential variables are replaced by Skolem constants or functions of the enclosing universal variables, universal quantifiers are dropped, and $\vee$ is distributed over $\wedge$. A clause is a set of signed literals $(sign, predicate, args)$, terms being constants, variables or function applications. Equality is axiomatised by reflexivity and symmetry. Because hand-written annotations contain typographical faults, the parser is tolerant: unbalanced brackets are repaired by the most plausible insertion or deletion, malformed premises are skipped (which can only weaken the KB), and near-identical predicate or constant names (string similarity ≥ 0.85) are merged.

3.3 Structural causal models

A sentence such as "A and B together cause C", "A or not B causes C" or "A causes not B and D" is turned into a Boolean structural equation for each effect variable, the right-hand side being a conjunction or disjunction of (possibly negated) causes; several equations for the same effect are OR-combined. Variables without a cause are exogenous; a stated Bernoulli prior p is attached to them, otherwise a uniform 0.5 is used. Observations become evidence literals, and the question is parsed into a set of interventions $\mathrm{do}(V = value)$ and a target variable.

4 Reasoning algorithms

4.1 Forward and backward chaining

Forward chaining repeatedly fires every rule whose body is satisfied (matching rule variables to constants by unification) and adds the head to the set of known atoms until a fixed point or until the query is derived. It is sound and complete for Horn rules, runs in time polynomial in the size of the KB, and records for each derived atom the rule and premises that produced it, which gives a proof tree for free. Backward chaining starts from the query and recursively tries to establish the body of each rule whose head unifies with the current goal. It is also exposed as an explicit state-space search: a state is the set of open sub-goals, a move replaces one sub-goal by the body of a matching rule (or discards it if it is a known fact), and the goal state is the empty set. The frontier is managed in four ways: BFS (FIFO), DFS (LIFO), uniform-cost search (priority $g =$

number of rules used) and A* (priority $g + h$, with h the number of open sub-goals, an admissible estimate since each open sub-goal needs at least one further step). A visited set prevents loops through cyclic rules.

4.2 Unification and resolution refutation

Unification computes a most general unifier of two terms with an occurs check. To test $K \models q$, the clauses of K and of $\neg q$ are combined and the resolution procedure searches for the empty clause (refutation completeness). A given-clause loop is used: the best clause is removed from a priority queue, resolved against all clauses with a complementary literal after renaming variables apart, and factored; new clauses pass tautology deletion, duplicate detection (canonical form up to variable renaming) and size bounds (literals per clause, term depth) before they are queued. The queue order is the search strategy: BFS orders by derivation depth, DFS by recency, and A* by $f = g + h$ where g is the derivation depth and h the total symbol size of the clause, so that short, specific clauses (which are close to the empty clause) are expanded first. A time limit and a cap on expanded clauses bound the effort; when they are reached the query is reported as not entailed.

4.3 Ground DPLL

Over a finite universe, first-order entailment reduces to propositional unsatisfiability. The clause set is grounded over the Herbrand universe (all constants occurring in the problem plus Skolem terms up to a small depth), every ground atom receives an integer identifier, and the DPLL procedure of Davis, Putnam, Logemann and Loveland is applied. DPLL combines unit propagation, pure-literal elimination and branching on the most frequent variable. If the ground set of $K \wedge \neg q$ is unsatisfiable, entailment is proved (this direction is sound regardless of the depth limit); if the grounding would be too large, control passes to resolution. Running the cheap SAT test first reduces total time considerably (Section 5.3).

4.4 Counterfactual inference in causal models

Counterfactual questions are answered in Pearl's three steps. *Abduction:* every assignment of the exogenous variables is enumerated (a complete depth-first enumeration, since the graphs are small), weighted by the product of the priors, and kept only if the factual world it induces agrees with the observations and with the stated factual value of the intervened variable ("instead of X"). *Action:* in the surviving worlds the intervened variables are overwritten. *Prediction:* the equations are propagated forward (a fixed-point evaluation that also tolerates cyclic mechanisms) and the target's value is read off. The answer is *yes* if the weighted probability of the target being true exceeds 0.5.

5 Experiments on the public data

5.1 Protocol

All public material was used for development and evaluation: RuleTaker (train 1,000; test 250; out-of-distribution test 375, with deeper proofs of depth 4–5), FOLIO v2 (train 1,001; validation 203) and CounterBench (alpha, 1,000 questions of four types; backdoor, 200 questions with confounders). For FOLIO the three-way label is collapsed: True stays True, while False and Uncertain both count as not entailed (False). The private test set was never used. Accuracy is the share of correct predictions; efficiency is the wall-clock time spent inside the reasoning engines on a standard laptop-class CPU, parsing included.

5.2 Overall results

Data set	Correct / total	Accuracy	Time (s)
RuleTaker – train	983 / 1000	98.30 %	0.27
RuleTaker – test	244 / 250	97.60 %	0.06
RuleTaker – OOD test (depth 4–5)	373 / 375	99.47 %	0.09
FOLIO – validation (FOL given)	177 / 203	87.19 %	6.5
FOLIO – train (FOL given)	803 / 1001	80.22 %	19.6
CounterBench – alpha	960 / 1000	96.00 %	0.13
CounterBench – backdoor	180 / 200	90.00 %	0.02
Overall	**3720 / 4029**	**92.33 %**	**≈27**

Table 2: Accuracy and total inference time on every public split.

Rule-based reasoning is both the most accurate and by far the cheapest (about 0.3 ms per query). Accuracy does not deteriorate on the out-of-distribution split whose proofs are deeper than any seen in the training split, which is the behaviour expected of an engine that is complete for Horn rules rather than one fitted to data. More than 95% of the total running time is spent on the 1,204 FOLIO items.

Collection	Breakdown (correct / total)
RuleTaker train	depth 1: 196/196 · depth 2: 393/406 · depth 3: 394/398
RuleTaker test	depth 1: 54/54 · depth 2: 89/94 · depth 3: 101/102
RuleTaker OOD	depth 4: 250/250 · depth 5: 123/125
CounterBench alpha	basic: 250/250 · conditional: 230/250 · joint: 240/250 · nested: 240/250
CounterBench backdoor	180/200 (confounded counterfactuals)
FOLIO validation	True: 53/72 · False: 57/62 · Uncertain: 67/69
FOLIO train	True: 240/388 · False: 261/289 · Uncertain: 302/324

Table 3: Accuracy by proof depth, question type and gold label.

5.3 Comparison of search strategies

Backward-chaining proof search was run with the four frontier policies on the RuleTaker test split, and the first-order engines were compared on FOLIO validation with the same 0.5 s per-problem limit.

Several observations follow. (i) The proof trees in RuleTaker are shallow and narrow, so all four frontier policies find the same proofs and differ only marginally in node count; A* and UCS match BFS because every move has unit cost, and DFS expands slightly more nodes. (ii) For resolution, the informed A* ordering is clearly better than blind BFS or DFS: it proves more theorems (+2.95 points) in less time, because the weight heuristic steers the search to short clauses. (iii) Grounded DPLL is the cheapest FOLIO engine, since the stories involve few constants and the ground clause sets are small; this is why it is run first and resolution serves as a fallback for problems whose grounding is too large or that require deeper Skolem terms. The two first-order variants differ by a single problem.

Method	Accuracy	Cost
Backward chaining – BFS	97.60 %	2.9 nodes / query, 0.02 s total
Backward chaining – DFS	97.60 %	3.3 nodes / query, 0.02 s total
Backward chaining – UCS	97.60 %	2.9 nodes / query, 0.02 s total
Backward chaining – A*	97.60 %	2.9 nodes / query, 0.02 s total
Resolution – A* clause selection	87.19 %	10.98 s
Resolution – BFS clause selection	84.24 %	14.95 s
Resolution – DFS clause selection	84.24 %	15.77 s
Ground DPLL	87.68 %	1.51 s

Table 4: Search strategies for rule-base proofs (RuleTaker test) and first-order entailment (FOLIO validation).

6 Examples of reasoning traces

Trace A – forward chaining with a proof tree. KB: *Bob is young. Bob is kind. If someone is young, then they are smart. If someone is smart and kind, then they are successful.* Query: *Bob is successful.*

```
successful(bob)   [by rule: If someone is smart and kind, then they are
                   successful.]
  smart(bob)      [by rule: If someone is young, then they are smart.]
    young(bob)    [given fact]
  kind(bob)       [given fact]
=> True
```

For the query *Bob is rich.* the fixed point contains *young, kind, smart, successful* only; `rich(bob)` is not derivable and the answer is False (negation as failure).

Trace B – resolution refutation. Premises: $\forall x\,(Man(x) \to Mortal(x))$, $Man(socrates)$. Query: $Mortal(socrates)$.

```
clauses      : Mortal(?v) ∨ ¬Man(?v)      Man(socrates)
negated goal : ¬Mortal(socrates)
step 1  resolve  ¬Mortal(socrates)  with  Mortal(?v) ∨ ¬Man(?v)
        {?v := socrates}  ->  ¬Man(socrates)
step 2  resolve  ¬Man(socrates)  with  Man(socrates)
        ->  [] (empty clause)  =>  KB ∧ ¬query is unsatisfiable  =>  True
```

Trace C – counterfactual query. Story: *Blaf causes Ziklo, Ziklo or Blaf causes Trune, Trune causes Vork, Vork causes Lumbo. Blaf ∼ Bern(0.2). We observed Trune.* Question: *Would Lumbo occur if not Ziklo instead of Ziklo?*

```
mechanisms  : Ziklo := Blaf ; Trune := Ziklo ∨ Blaf ; Vork := Trune ;
```

```
              Lumbo := Vork
evidence    : Trune = 1 ;  factual value of the intervened variable: Ziklo = 1
abduction   : Ziklo = 1 forces Blaf = 1 (the only world consistent with the evidence)
action      : do(Ziklo = 0)   (Blaf stays 1, its value is a property of the world)
prediction  : Trune = 0 ∨ 1 = 1  ->  Vork = 1  ->  Lumbo = 1
P(Lumbo = 1) = 1.0 > 0.5  =>  yes (True)
```

Trace C illustrates why a naïve forward simulation (setting Ziklo to false and propagating) would be wrong: the confounder Blaf, inferred in the abduction step, still activates Trune.

7 Conclusion

A combination of classical techniques—chaining over Horn rules, informed resolution with unification, ground DPLL, and exhaustive abduction–action–prediction over causal models—solves entailment queries from three different families of benchmarks with 92.3% accuracy on the public data and with low computational effort. The experiments show that (a) exact, complete procedures are essentially error-free when the representation is faithful, (b) heuristic search (A*) is worth its overhead only where the search space is large, namely in first-order resolution, and (c) the dominant source of remaining error is the quality of the translation into logic, not the reasoning itself.

References

[1] Russell, S., & Norvig, P. (2021). *Artificial Intelligence: A Modern Approach* (4th ed.). Pearson.